

INF/UFRGS – Computacao Paralela | 2026-05-27

Paralelizacao do Algoritmo Q-Learning com OpenMP

Rayan Raddatz de Matos, Eduardo Magnus Lazuta, Enzo Lisboa Peixoto



Agenda

- Problema e Algoritmo
- Paralelizacao
- Metodologia Experimental
- Resultados
- Conclusao

Problema e Algoritmo

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ ate $(L - 1, C - 1)$ em um grid $L \times C$ com obstaculos. Acoes: cima, baixo, esquerda, direita.

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ ate $(L - 1, C - 1)$ em um grid $L \times C$ com obstaculos. Acoes: cima, baixo, esquerda, direita.

A cada passo, a Q-Table e atualizada pela equacao de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ ate $(L - 1, C - 1)$ em um grid $L \times C$ com obstaculos. Acoes: cima, baixo, esquerda, direita.

A cada passo, a Q-Table e atualizada pela equacao de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Apos o treinamento, a politica otima e extraida como $\pi(s) = \arg \max_a Q(s, a)$.

Estrutura de um episodio

- Parte do estado inicial s_0
- Seleciona acao via epsilon-greedy
- Transita para s' , recebe recompensa r
- Atualiza $Q(s, a)$ pela equacao de Bellman
- Repete ate atingir o objetivo ou esgotar passos

Recompensas configuradas:

- $R_{\text{objetivo}} = +100$
- $R_{\text{obstaculo}} = -100$
- $R_{\text{passo}} = -1$
- $R_{\text{loop}} = -10$

Grids testados: **10x10, 50x50, 100x100**

Paralelizacao

Qual nivel de paralelismo?

Tres opcoes foram consideradas:

- **Acoes** (argmax de 4 acoes): descartada -- overhead supera o ganho

Qual nivel de paralelismo?

Tres opcoes foram consideradas:

- **Acoes** (argmax de 4 acoes): descartada -- overhead supera o ganho
- **Steps** (passos do episodio): descartada -- cada step depende do anterior

Qual nivel de paralelismo?

Tres opcoes foram consideradas:

- **Acoes** (argmax de 4 acoes): descartada -- overhead supera o ganho
- **Steps** (passos do episodio): descartada -- cada step depende do anterior
- **Episodios: escolhida** -- episodios sao independentes; distribuicao direta via omp for

Qual nivel de paralelismo?

Tres opcoes foram consideradas:

- **Acoes** (argmax de 4 acoes): descartada -- overhead supera o ganho
- **Steps** (passos do episodio): descartada -- cada step depende do anterior
- **Episodios: escolhida** -- episodios sao independentes; distribuicao direta via omp for

Mas paralelizar sem sincronizacao nao basta: com T threads e E episodios, cada thread aprende como se tivesse apenas E/T episodios.

Sincronizacao Periodica

Cada thread mantem uma **Q-Table local**, treina um lote de `sync_interval` episodios e funde os deltas na Q-Table global:

```
#pragma omp for schedule(runtime)
for (episode = ep_start; episode < ep_end; episode++)
    run_episode(&local_ql, episode);

#pragma omp for collapse(2)
for (state ...) for (action ...) {
    delta_sum = sum_t(local[t][s][a] - snapshot);
    ql->q_table[state][action] = snapshot + delta_sum / active;
}
```

Merge por delta: divide pelas threads **ativas** ($\text{active} = \min(\text{chunk}, T)$), preservando o α efetivo quando $\text{chunk_size} < T$.

Heurísticas de `sync_interval`

Duas estrategias automaticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergencia local (inspirado em limites de regret para SGD paralelo)

Heurísticas de `sync_interval`

Duas estratégias automáticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergencia local (inspirado em limites de regret para SGD paralelo)
- **statespace**: `sync_interval = L * C` -- cada thread explora todos os estados antes de fundir

Heurísticas de `sync_interval`

Duas estratégias automáticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergencia local (inspirado em limites de regret para SGD paralelo)
- **statespace**: `sync_interval = L * C` -- cada thread explora todos os estados antes de fundir

Alem disso, testamos valores fixos: **every10** (`i=10`) e **every1000** (`i=1000`).

HOGWILD!

Todas as threads leem e escrevem na **mesma Q-Table sem nenhum lock**:

```
QLearning local_ql = *ql;  
local_ql.rand_seed = seed + tid;  
/* q_table aponta para a tabela COMPARTILHADA */  
  
#pragma omp for schedule(runtime)  
for (episode = 0; episode < N; episode++)  
    run_episode(&local_ql, episode); /* sem lock */
```


HOGWILD!

Todas as threads leem e escrevem na **mesma Q-Table sem nenhum lock**:

```
QLearning local_ql = *ql;  
local_ql.rand_seed = seed + tid;  
/* q_table aponta para a tabela COMPARTILHADA */  
  
#pragma omp for schedule(runtime)  
for (episode = 0; episode < N; episode++)  
    run_episode(&local_ql, episode); /* sem lock */
```

Race conditions sao **intencionais e inofensivas** porque:

- **Esparsidade**: cada episodio visita fracao pequena dos $L \times C$ estados
- **Escala**: updates ponderados por $\alpha \leq 0.1$ -- sobrescrever um e ruido $O(\alpha)$

- **Simetria:** ambos os valores concorrentes sao gradientes validos

Ref: Niu et al., **Hogwild!**, NIPS 2011.

Metodologia Experimental

Fatores do experimento

Fator	Niveis	Motivacao
Threads	1, 2, 4, 8, 16, 20, 40	Do serial ao hyperthreading completo
Tamanho grid	10, 50, 100	Espaco de estados pequeno/medio/grande
Modo de sync	sqrt, statespace, hogwild, 10, 1000	5 estrategias de paralelizacao
Episodios	5000, 10000	5k instavel, 10k converge melhor
Obstaculos	0%, 50%	Grid livre vs. alta densidade
Schedule OMP	static, guided, dynamic	Políticas de distribuicao
Chunk	0 (auto), 1, 10	Granularidade do chunking

Baseline: binario sequencial puro (sem runtime OpenMP) para $T = 1$.

Afinidade de threads e medicao de tempo

Afinidade:

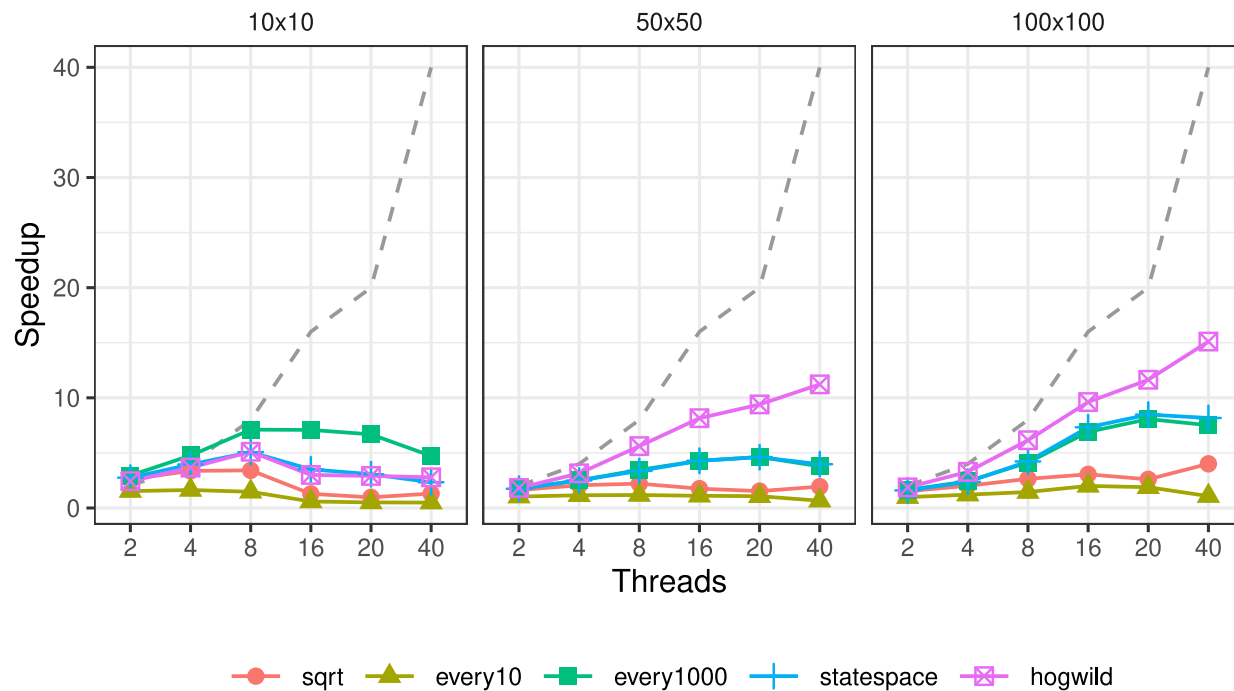
- $T \leq 20$: `OMP_PLACES=cores, OMP_PROC_BIND=close` -- 1 thread por nucleo fisico
- $T = 40$: `OMP_PLACES=threads` -- hyperthreading completo

Medicao de tempo:

- Apenas o tempo de `train()` -- exclui alocao, init e I/O
- Paralelo: `omp_get_wtime()` | Sequencial: `clock_gettime(CLOCK_MONOTONIC)`
- Multiplos blocos de replicacao para estimar variancia do escalonador

Resultados

Speedup por modo de sincronizacao



Speedup: observacoes

- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead

Speedup: observacoes

- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead
- **100x100**: HOGWILD! atinge ate **17x com 40 threads** modos sincronizados ficam abaixo de 10x

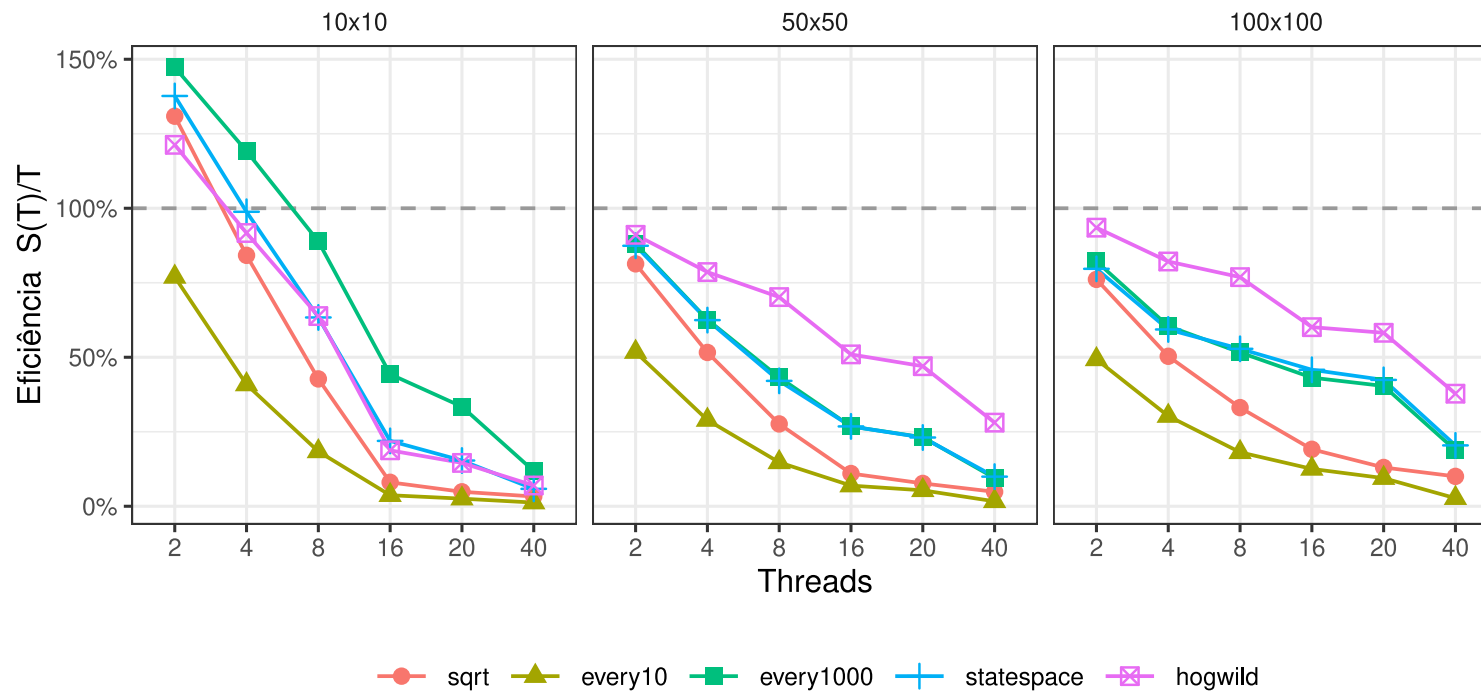
Speedup: observacoes

- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead
- **100x100**: HOGWILD! atinge ate **17x com 40 threads** modos sincronizados ficam abaixo de 10x
- Barreiras periodicas serializam parcialmente a execucao nos modos sincronizados

Speedup: observacoes

- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead
- **100x100**: HOGWILD! atinge ate **17x com 40 threads** modos sincronizados ficam abaixo de 10x
- Barreiras periodicas serializam parcialmente a execucao nos modos sincronizados
- Grid 10x10 com T=2: eficiencia **super-linear** (efeito de cache com Q-Table de 3,2 KB)

Eficiencia paralela $S(T)/T$



Eficiencia: observacoes

- HOGWILD! mantem a **melhor eficiencia** em todos os grids: 35--45% com 40 threads no 100x100

Eficiencia: observacoes

- HOGWILD! mantem a **melhor eficiencia** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiencia** -- overhead de barreiras frequentes supera o ganho

Eficiencia: observacoes

- HOGWILD! mantem a **melhor eficiencia** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiencia** -- overhead de barreiras frequentes supera o ganho
- every1000 e statespace ficam no intermediario: sincronizam menos e amortizam o custo

Eficiencia: observacoes

- HOGWILD! mantem a **melhor eficiencia** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiencia** -- overhead de barreiras frequentes supera o ganho
- every1000 e statespace ficam no intermediario: sincronizam menos e amortizam o custo
- Nenhuma configuracao sustenta 50% de eficiencia alem de 8 threads no grid 10x10

Analise de Cache (10x10)

Q-Table do 10x10 ocupa **3,2 KB** (100 estados x 4 acoes x 8 bytes) -- cabe inteira em L1.

Taxa de LLC-miss medida com perf stat:

Threads	sqrt	statespace	hogwild
2	0,80%	1,11%	0,44%
4	0,70%	0,79%	0,18%
8	0,17%	0,18%	0,11%
16	30,9%	23,1%	36,7%
20	29,1%	22,9%	36,4%
40	40,7%	28,5%	35,9%

Ponto de ruptura em $T \in [8, 16]$ confirma: gargalo e **tráfego de cache**, não falta de trabalho.

Analise de Cache: interpretacao

Para $T \leq 8$: hogwild tem LLC-miss **menor** -- Q-Table unica de 3,2 KB fica estavel em L1/L2. Modos federados tem copias separadas por thread, aumentando o footprint.

Analise de Cache: interpretacao

Para $T \leq 8$: hogwild tem LLC-miss **menor** -- Q-Table unica de 3,2 KB fica estavel em L1/L2. Modos federados tem copias separadas por thread, aumentando o footprint.

Para $T \geq 16$: hogwild passa a ter **mais** LLC-miss (36,7%) -- cada escrita invalida a linha de cache nos demais núcleos (protocolo MESI). Statespace aguenta melhor (23,1%) pois Q-Tables locais ficam isoladas por mais tempo entre syncs.

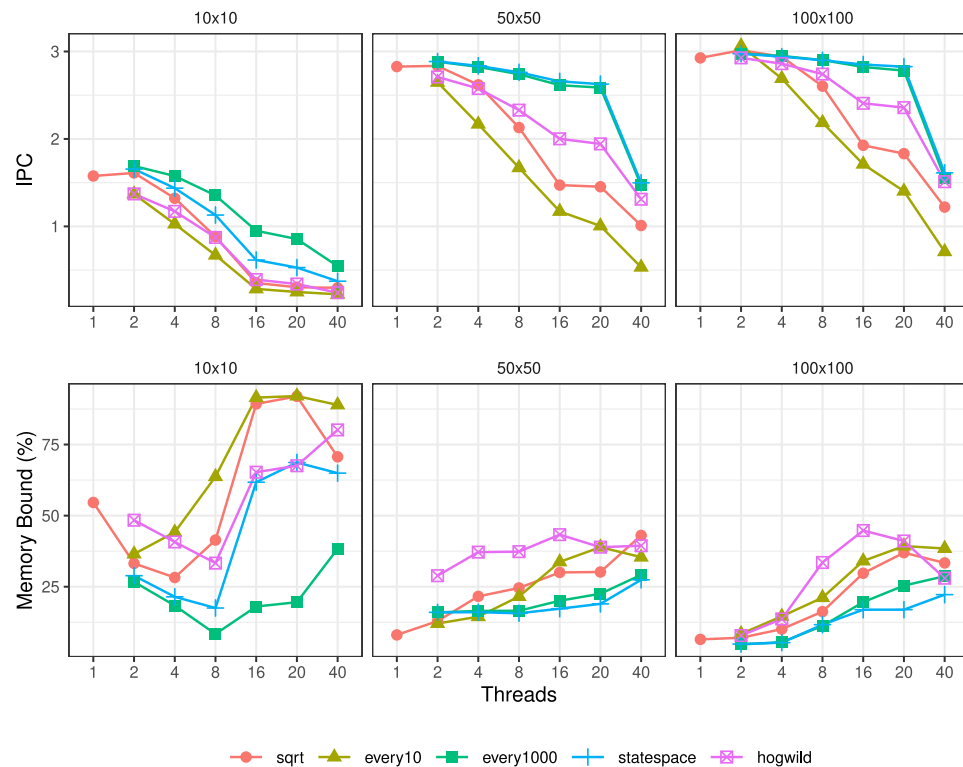
Analise de Cache: interpretacao

Para $T \leq 8$: hogwild tem LLC-miss **menor** -- Q-Table unica de 3,2 KB fica estavel em L1/L2. Modos federados tem copias separadas por thread, aumentando o footprint.

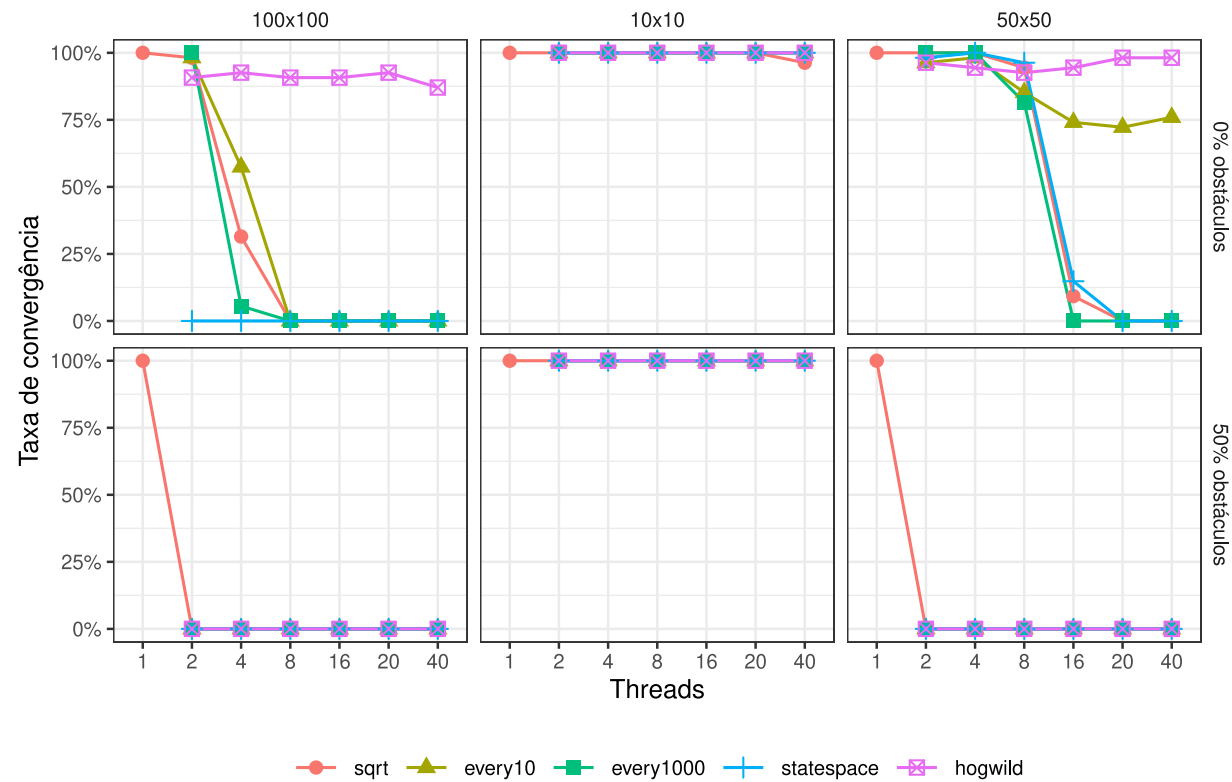
Para $T \geq 16$: hogwild passa a ter **mais** LLC-miss (36,7%) -- cada escrita invalida a linha de cache nos demais núcleos (protocolo MESI). Statespace aguenta melhor (23,1%) pois Q-Tables locais ficam isoladas por mais tempo entre syncs.

Confirmado pelo VTune: **Memory Bound** alto e **IPC baixo** no 10x10 com muitas threads.

VTune: IPC e Memory Bound



Convergencia



Convergencia: causa da degradacao

Com T threads e E episodios fixos, cada thread treina com E/T episodios. Quando $E/T < E_{\min}$ o agente nao aprende.

Convergencia: causa da degradacao

Com T threads e E episodios fixos, cada thread treina com E/T episodios. Quando $E/T < E_{\min}$ o agente nao aprende.

- **0% de obstaculos:** todos os modos convergem para poucos threads; degrada com o aumento
- **50% de obstaculos:** nenhuma configuracao paralela convergiu para grids grandes

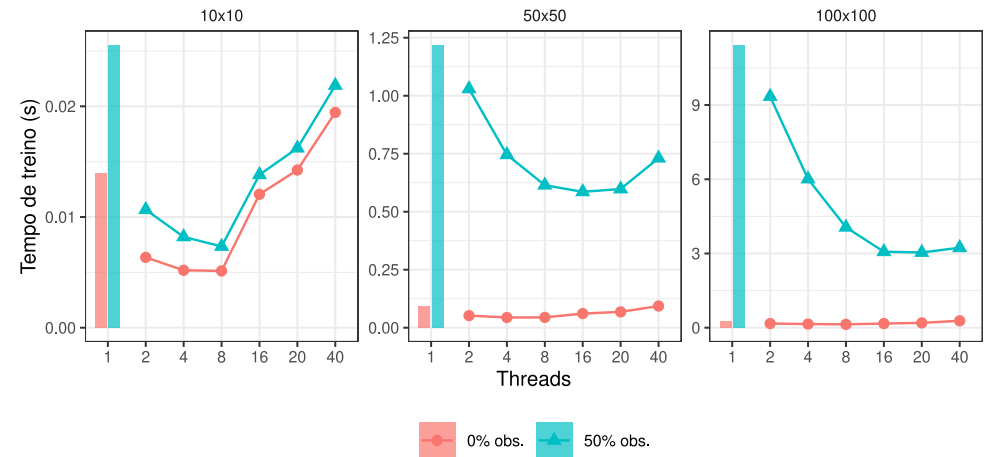
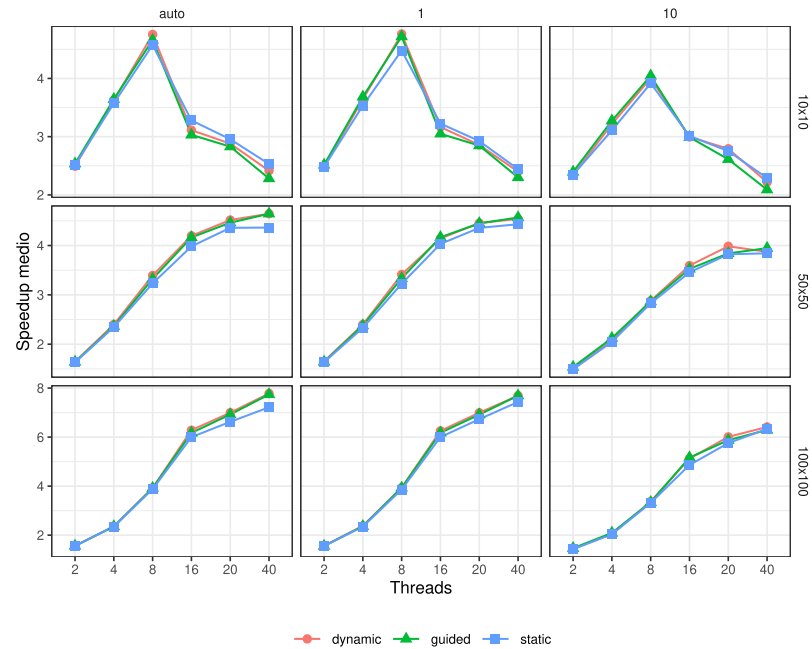
Convergencia: causa da degradacao

Com T threads e E episodios fixos, cada thread treina com E/T episodios. Quando $E/T < E_{\min}$ o agente nao aprende.

- **0% de obstaculos:** todos os modos convergem para poucos threads; degrada com o aumento
- **50% de obstaculos:** nenhuma configuracao paralela convergiu para grids grandes

Mesmo HOGWILD! (que acumula updates de **todas** as threads) falha com 50% de obstaculos: race conditions corrompem o aprendizado em ambientes densos.

Schedule, chunk e obstaculos



Schedule e chunk: nao sao fatores dominantes

- Os tres schedules produzem speedup **praticamente identico**: duracoes de episodio homogeneas, carga equilibrada independente da politica
- Diferenca geral permanece **<2% no speedup medio**
- Chunk=1: leve degradacao em grids pequenos (overhead por episodio); chunk=10 e auto se comportam de forma equivalente
- Obstaculos: comparar speedup entre 0% e 50% e **injusto** -- ganho aparente com 50% reflete apenas que ha mais trabalho total a dividir

Conclusao

Conclusao

- **Gargalo principal:** divisao de episodios. Com T threads e E fixos, cada thread treina com E/T -- abaixo do limiar, agente nao converge
- **Grid 10x10:** gargalo e coerencia de cache -- LLC-miss explode de $<1,2\%$ para 23--40% entre $T = 8$ e $T = 16$
- **HOGWILD!** obteve maior speedup bruto (ate **17x** com 40 threads em 100x100), mas nao converge com 50% de obstaculos
- Modos sincronizados com intervalos longos (**every1000**, **statespace**) preservam melhor o comportamento de cache entre syncs, mas convergem menos

- Schedule e chunk: **nao sao fatores dominantes** (<2% de impacto no speedup)

Perguntas?